MyCodeSchool

Liste: din C++ în Python

Partea 2 · David · teorie

Vectorul din C++ se numește **listă** în Python. Algoritmii sunt aceiași, dar lista are două lucruri pe care `int v[100]` nu le are: **știe câte elemente are** și **poate crește**. Tot în **două stiluri**: **A — ca în C++** (indici, `for i in range(len(v))`, comparații scrise de tine) și **B — stil Python** (uneltele din secțiunea 4).

1. Același program, în ambele limbaje

main.cpp

```cpp
#include <iostream>
using namespace std;

int suma_elementelor(int v[], int n)
{
    int s = 0;
    for (int i = 0; i < n; i++)
        s += v[i];
    return s;
}

int main()
{
    int v[100], n;
    cin >> n;
    for (int i = 0; i < n; i++)
        cin >> v[i];
    cout << suma_elementelor(v, n);
    return 0;
}
```

main.py — stil A

```python
def suma_elementelor(v: list) -> int:
    s = 0
    for i in range(len(v)):
        s = s + v[i]
    return s


def main() -> None:
    v = [int(x) for x in input().split()]
    print(suma_elementelor(v))


if __name__ == "__main__":
    main()
```

C++	Python
`int v[100];` — mărime fixă, aleasă de tine	`v = []` — crește cât e nevoie, nu declari mărimea
n se citește și se plimbă peste tot ca parametru	`len(v)` — lista își știe singură lungimea, **nu mai ai nevoie de n**
`cin >> n;` apoi `for` cu `cin >> v[i];`	`v = [int(x) for x in input().split()]` — toate numerele de pe o linie, despărțite prin spațiu
`for (int i = 0; i < n; i++)`	`for i in range(len(v)):`
`suma_elementelor(v, n)`	`suma_elementelor(v)` — un singur parametru
`cout << v[i] << " ";` într-o buclă	`print(v)` — afișează `[7, 3, 10, 5]`, cu paranteze și virgule

`input().split()` taie linia tastată la spații și dă o listă de **texte**: `["7", "3"]`. De-aia e nevoie de `int(x)` pe fiecare. Dacă vrei fiecare număr pe linia lui, ca în C++: `v = [int(input()) for _ in range(n)]`.

2. Lista știe câte elemente are

Pentru `v = [7, 3, 10, 5]`:

Scrii	Obții	În C++
`len(v)`	4	n, pe care îl purtai cu tine
`v[0]`, `v[3]`	7, 5	la fel
`v[-1]`, `v[-2]`	5, 10 — ultimul, penultimul	`v[n-1]`, `v[n-2]`
`v[4]`	**IndexError** — programul se oprește	citește gunoi din memorie, fără să te anunțe
`for x in v:`	7, 3, 10, 5 — elementele, fără indice	nu există; mereu prin indici
`range(4)`	0, 1, 2, 3 — **fără 4**	`for (i = 0; i < 4; i++)`
`range(1, 4)`	1, 2, 3	`for (i = 1; i < 4; i++)`
`range(3, -1, -1)`	3, 2, 1, 0 — invers	`for (i = 3; i >= 0; i--)`

Când ai nevoie și de poziție și de valoare: `for i in range(len(v)):` și apoi `v[i]`. Stilul A folosește mereu forma asta, ca în C++.

3. Lista se modifică — `int` nu

În Partea 1 ai văzut că un `int` dat ca parametru nu poate fi schimbat pentru cel care a apelat funcția. **La listă e invers:** funcția primește aceeași listă, nu o copie, deci dacă o modifică, modificarea se vede și afară. E ca `int v[]` din C++, care se transmite prin adresă.

C++ — mutarea elementelor o faci tu

```
// adaugi 8 la final
v[n] = 8;
n++;

// stergi elementul de pe poz. 1
for (int i = 1; i < n - 1; i++)
    v[i] = v[i + 1];
n--;

// copiezi vectorul
int b[100];
for (int i = 0; i < n; i++)
    b[i] = v[i];
```

Python — lista o face singură

```
v = [7, 3, 10, 5]

v.append(8)      # [7, 3, 10, 5, 8]
v.insert(1, 99)  # [7, 99, 3, 10, 5, 8]
del v[1]         # [7, 3, 10, 5, 8]
v.remove(10)     # scoate prima aparitie a lui 10
ultimul = v.pop()  # scoate si intoarce ultimul

b = v            # ATENTIE: acelasi obiect
b = v[:]         # copie adevarata
b = list(v)      # la fel, copie
```

`b = v` nu copiază nimic: cele două nume ajung la aceeași listă, deci `b.append(1)` schimbă și `v`. În C++ `int b[100]` era de la început altă zonă de memorie; în Python ceri copia explicit, cu `v[:]` sau `list(v)`.

De aceea, în toate problemele din fișă, **funcțiile întorc o listă nouă și nu strică lista primită**: construiești `rez = []` și adaugi în ea cu `rez.append(...)`. Ca să schimbi două elemente între ele n-ai nevoie de variabilă temporară: `v[i], v[j] = v[j], v[i]`.

4. Stilul B: uneltele pe liste

Pentru `v = [7, 3, 10, 5, 3]`:

Scrii	Obții	Echivalentul din stilul A
`sum(v)`	28	buclă cu `s = s + v[i]`
`max(v)`, `min(v)`	10, 3	buclă cu `if v[i] > mx`
`v.index(3)`	1 — **prima** apariție	buclă care se oprește la primul egal
`v.count(3)`	2	buclă cu contor
`3 in v`, `8 not in v`	`True`, `True`	căutare cu buclă
`sorted(v)`	`[3, 3, 5, 7, 10]` — **listă nouă**	sortare prin selecție, scrisă de tine
`sorted(v, reverse=True)`	`[10, 7, 5, 3, 3]`	aceeași sortare, cu < în loc de >
`v[::-1]`	`[3, 5, 10, 3, 7]`	buclă de la `len(v)-1` în jos
`v[1:3]`, `v[2:]`, `v[:2]`	`[3, 10]`, `[10, 5, 3]`, `[7, 3]`	copiere parțială cu buclă
`a + b`	o listă cu elementele lui a, apoi ale lui b	două bucle de append
`[x for x in v if x % 2 == 0]`	`[10]`	buclă cu `if` și append
`[x * 2 for x in v]`	`[14, 6, 20, 10, 6]`	buclă care construiește altă listă

`[x for x in v if x % 2 == 0]` se citește „lista lui x, pentru fiecare x din v, dacă x e par”. Aceeași formă ca `sum(int(c) for c in str(n))` din Partea 1 — doar că rezultatul e o listă, nu o sumă.

5. Capcane

`v.sort()` nu întoarce nimic: `v = v.sort()` îți lasă în v valoarea `None`, și următoarea linie crapă: `v.append(1)` dă `AttributeError`, iar `v[0]` sau `len(v)` dau `TypeError`. `v.sort()` rearanjează lista pe loc; dacă vrei o listă nouă, folosești `sorted(v)`.

Ștergerea în timpul parcurgerii: dacă ștergi din listă cât timp o parcurgi, elementele se mută la stânga și sari peste unul. Construiești o listă nouă cu `append` — e și mai aproape de ce făceai în C++.

`input().split()` dă texte: fără `int(x)`, v conține `["7", "3"]`, iar `max(v)` compară alfabetic: `"9"` iese mai mare decât `"10"`.

Lista goală: `v[0]` și `max(v)` crapă pe o listă fără elemente. În problemele din fișă lista are cel puțin un element — dar la bac verifici întâi `if len(v) == 0:`.